

Folien-Analyse — Sicherheit / Datenschutz (Säule 6)

Quelle: datenschutz/Sicherheit und Privacy.pdf (Kern) · Web Engineering_Day6.pdf → **Dozent: davon nur SEO klausurrelevant** (Rest nicht; Details im Day6-Anhang unten). Gegen OWASP/MDN verifiziert. Themen-IDs D1–D4. Format: Q&A + Kontrast-Tabellen.

Grundlagen (Querschnitt)

- **Security ≠ Privacy:** Sicherheit = Schutz von Systemen/Daten vor unbefugtem Zugriff/Manipulation/Ausfall ("Wer darf was, wie zuverlässig?"). Privacy = Schutz personenbezogener Daten / informationelle Selbstbestimmung ("Welche Daten nötig, wer nutzt sie wofür?"). Eine App kann technisch sicher und trotzdem datenschutzwidrig sein.
- **Schutzziele (CIA):** Confidentiality (Vertraulichkeit), Integrity (Integrität), Availability (Verfügbarkeit). Ergänzend: Authentizität, Nachvollziehbarkeit, Datenminimierung, Zweckbindung.
- **Authentifizierung vs. Autorisierung:**

Frage	Authentifizierung	Autorisierung
Worum	Identität nachweisen	Berechtigung prüfen
Mittel	Passwort, MFA, Passkey	Rollen, Rechte, Scopes, Policies
Beispiel	Login	darf Rechnung lesen, nicht löschen

- **OAuth2:** Framework für **delegierte Autorisierung** — App bekommt begrenzten Zugriff auf Ressourcen eines anderen Dienstes, ohne Passwort weiterzugeben. Ablauf: Client fordert Zugriff → User bestätigt beim Authorization Server → Token mit Scopes → API prüft Token. **Löst Autorisierung, nicht Identität.**
- **OpenID Connect (OIDC):** Identity Layer auf OAuth2 — fügt ID Token + Claims (`sub` , `email` , `name`) hinzu ("Login mit Google/GitHub", SSO).
- **JWT:** `Header.Payload.Signature` , Base64Url-kodiert (NICHT verschlüsselt!). Regeln: keine Geheimnisse im Payload, kurze Laufzeiten, Signatur prüfen, Scopes minimal, Revocation/Logout mitdenken.

D1 — Typische Angriffe: XSS, CSRF, Injection (Definition/MC, mittel)

XSS (Cross-Site Scripting)

Bei XSS schleust ein Angreifer fremdes JavaScript in eine Seite ein, das im Browser anderer Nutzer läuft:

- **Definition:** Angreifer schleust JS/HTML ein, das im Browser anderer Nutzer ausgeführt wird.
- **Arten:** Stored (gespeichert, später ausgeliefert), Reflected (über Request-Parameter zurück), DOM-XSS (unsichere Frontend-JS-Verarbeitung).
- **Gegenmaßnahmen:** kontextabhängiges Output-Escaping, keine unsicheren DOM-Sinks (`innerHTML`), CSP als zusätzliche Schicht.

CSRF (Cross-Site Request Forgery)

- **Definition:** Eingeloggter Nutzer besucht bössartige Seite, die im Hintergrund einen Request an die echte App auslöst (Cookies werden automatisch mitgesendet).
- **Abwehr:** CSRF-Token für zustandsändernde Requests, `SameSite` -Cookies, Origin/Referer-Prüfung, kritische Aktionen bestätigen lassen.

Injection

- **Definition:** Daten werden zu Befehlen (SQL, NoSQL, Command, LDAP/XPath).
- **Abwehr: parametrisierte Queries** (statt String-Konkatenation), Allow-Lists, strikte Trennung Daten/Befehle, DB-/Systemrechte minimieren.

Kontrast

Angriff	Kern	Hauptabwehr
XSS	fremdes Skript im Browser	Output-Escaping + CSP

Angriff	Kern	Hauptabwehr
CSRF	erzwungener Request mit gültiger Session	CSRF-Token + SameSite
Injection	Eingabe wird als Befehl ausgeführt	parametrisierte Queries

Fallen / Bugs

- **Falle:** `innerHTML` mit ungeprüfem Input = klassische XSS-Quelle.
- **Kontrast:** XSS nutzt Vertrauen des Nutzers in die Seite; CSRF nutzt Vertrauen der Seite in den Nutzer/Browser.

Drill

- [MC] `innerHTML = userInput` — welche Lücke?

Lösung: XSS.

- [MC] Welche Abwehr gegen SQL-Injection?

Lösung: parametrisierte Queries (Prepared Statements).

- [Definition] Wogegen hilft ein CSRF-Token?

Lösung: gegen erzwungene zustandsändernde Cross-Site-Requests.

D2 — HTTPS / TLS Grundlagen (Definition, mittel)

- **Leistet:** verschlüsselt Transport, prüft Server-Identität per Zertifikat, erschwert Mitlesen/Manipulation im Netz.
- **Löst NICHT:** XSS in der App, zu breite Rechte, unsichere Datenhaltung im Backend. → HTTPS ist Pflicht, aber nur Basiskontrolle.

OWASP Top Ten (Einordnung)

Die OWASP Top Ten listen die verbreitetsten Web-Sicherheitsrisiken; hier die klausurrelevante Einordnung:

- Keine vollständige Checkliste, sondern priorisierte Liste häufiger Web-Schwachstellen. Im Fokus: XSS, CSRF, Injection, Broken Access Control, Security Misconfiguration.
- **4 Engineering-Prinzipien:** Least Privilege (minimale Rechte), Secure by Default (eher blockieren), Validate on Trust Boundaries (Eingaben an jedem Übergang prüfen), Defense in Depth (mehrere Schutzebenen).

Fallen / Bugs

- **Falle:** "HTTPS = sicher" ist falsch — TLS schützt nur den Transportweg, nicht die Anwendungslogik.

Drill

- [MC] Schützt HTTPS vor XSS?

Lösung: Nein — nur den Transport; XSS bleibt App-Problem.

D3 — Datenschutz / DSGVO (Definition, niedrig)

- **DSGVO-Prinzipien:** Rechtmäßigkeit/Transparenz, Zweckbindung, Datenminimierung, Speicherbegrenzung, Integrität/Vertraulichkeit.
- **Für Entwickler:** nur benötigte Felder erheben, Lösch-/Exportprozesse einplanen, Third-Party-Dienste prüfen/dokumentieren.
- **Privacy by Design** (Datenschutz schon im Entwurf) vs. **Privacy by Default** (Voreinstellung = minimale Erhebung, Tracking erst nach Einwilligung).

Sichere Speicherung

Datentyp	Regel
Passwörter	nie Klartext, Hash mit Argon2/bcrypt

Datentyp	Regel
Schlüssel/Secrets	nicht im Quellcode, Secret Store/Env-Var
personenbezogene Daten	verschlüsseln, Zugriffe protokollieren, Aufbewahrung begrenzen

Fallen / Bugs

- **Falle:** Passwörter "verschlüsseln" ≠ hashen — korrekt ist **Hashing** (Argon2/bcrypt), nicht reversible Verschlüsselung.

Drill

- [MC] Wie Passwörter speichern?

Lösung: gehasht mit Argon2/bcrypt, nie im Klartext.

D4 — Sichere Cookies & Browser-Grenzen (MC/Definition, mittel)

Cookie-Flags

Diese Flags schränken ein, wann und wie ein Cookie gesendet und gelesen werden darf - zentrale Schutzmaßnahmen:

Flag	Funktion
HttpOnly	kein JS-Zugriff (document.cookie) → Schutz gegen XSS-Cookie-Diebstahl
Secure	nur über HTTPS übertragen
SameSite	steuert Mitsenden bei Cross-Site-Requests (Strict/Lax/None) → CSRF-Schutz
* Faustregel: Session-Cookie als HttpOnly; Secure; SameSite=Lax (bei nötigem Cross-Site None + CSRF-Schutz).	

Security-Header & Origin-Grenzen

- **CSP (Content Security Policy):** definiert vertrauenswürdige Quellen (default-src 'self'; script-src 'self' ...) → reduziert XSS-Wirkung. Erst report-only testen, keine Inline-Skripte.
- **CORS (Cross-Origin Resource Sharing):** Browser-Mechanismus; Server legt fest, welche fremden Origins zugreifen dürfen. **Regelt nur Browserzugriffe, schützt NICHT die API selbst.** Fehler: * zusammen mit Credentials, zu breite Allow-Listen, Verwechslung mit Authentifizierung.

Kontrast

Flag/Header	Schützt gegen
HttpOnly	XSS-Cookie-Auslese
Secure	Mitlesen über HTTP
SameSite	CSRF
CSP	XSS (Begrenzung ladbarer Ressourcen)
CORS	unkontrollierten Cross-Origin-Browserzugriff

Fallen / Bugs

- **Kontrast:** HttpOnly (kein JS) vs. Secure (nur HTTPS) vs. SameSite (Cross-Site-Verhalten) — oft als MC verwechselt.
- **Falle:** CORS ist kein API-Schutz, sondern eine Browser-Regel.

Drill

- [MC] Welches Cookie-Flag verhindert Auslesen per JavaScript?

Lösung: HttpOnly.

- [MC] Welches Flag/Mechanismus adressiert CSRF?

Lösung: `SameSite` -Cookie (+ CSRF-Token).

- [MC] Schützt CORS die API vor Angriffen?

Lösung: Nein — regelt nur Browser-Zugriffe zwischen Origins.

Anhang — Day6 (Usability/SEO/i18n/Patterns, NICHT Säule 6, knapp)

Dozenten-Aussage (Stand 07.07.2026): Aus `Web Engineering_Day6.pdf` ist **NUR SEO klausurrelevant**. WCAG, Responsive, Core Web Vitals, i18n/i10n, Entwurfsmuster aus Day6 → für die Klausur NICHT relevant (nur zur Einordnung gelistet). **SEO unbedingt können (↓ markiert).**

- **WCAG (Barrierefreiheit) — 4 Prinzipien:** Perceivable, Operable, Understandable, Robust. Level A < AA (Mindeststandard) < AAA. Semantik first, ARIA nur ergänzend ("kein ARIA besser als falsches ARIA").
- **Responsive:** Mobile-first, Viewport-Meta, fluid Units (% , vw , rem), Grid/Flexbox, `min-width` -Media-Queries, `srcset` / `<picture>`, `loading="lazy"`.
- **SEO — KLAUSURRELEVANT (Dozent).** Ziel: bessere Sichtbarkeit/Ranking in Suchmaschinen.
- **On-Page:** semantisches HTML (saubere Überschriften-Hierarchie h1→h6), aussagekräftiger `<title>` + `<meta name="description">`, sprechende URLs, `alt` -Texte für Bilder, sinnvolle interne Verlinkung.
- **Technisch:** `robots.txt` (steuert, was Crawler dürfen), `sitemap.xml` (URL-Liste für Suchmaschinen), `rel="canonical"` (Duplicate Content vermeiden), strukturierte Daten JSON-LD/Schema.org, `hreflang` (Sprach-/Länder-Varianten).
- **Off-Page:** Backlinks (Verweise anderer Seiten) als Reputationssignal.
- **Core Web Vitals:** LCP (< 2,5 s, Ladezeit größtes Element),